

Session 1 quiz — Introduction to Python for Finance

Ungraded in-class practice. Exam-style: code snippet + 5 true/false statements.

- **Review:** one question at a time
 - **Scope:** full DataCamp course (Ch 1–5)
 - **Plan:** 12 questions (Q11–Q12 are tricky review)
-

Question 1 (Chapter 1 — Variables and operators)

Consider the following code:

```
price = 120
earnings = 4
pe_ratio = price / earnings
print(pe_ratio)
```

- A. The variable `pe_ratio` will be equal to 30.0.
 - B. The type of `pe_ratio` is `int`.
 - C. In Python 3, `price / earnings` performs true division (not integer division).
 - D. `print(pe_ratio)` will display 30 without a decimal point.
 - E. Replacing `earnings = 4` with `earnings = "4"` would still produce `pe_ratio = 30.0`.
-

Question 2 (Chapter 1 — Data types and `type()`)

Consider the following code:

```
x = 5
y = 'stock'
print(x * 3)
print(y * 3)
```

- A. `print(x * 3)` outputs 15.
 - B. `type(x)` is `<class 'int'>`.
 - C. `print(y * 3)` outputs 'stockstockstock'.
 - D. The expression `y + 3` evaluates to 'stock3'.
 - E. Multiplying a string by an integer repeats the string that many times.
-

Question 3 (Chapter 2 — List indexing and slicing)

Consider the following code:

```
months = ['January', 'February', 'March', 'April', 'May', 'June']
a = months[0]
b = months[-1]
c = months[2:5]
```

- A. `a` is equal to 'January'.
- B. `b` is equal to 'June'.

- C. `c` is equal to `['March', 'April', 'May']`.
- D. `months[2:5]` includes the element at index 5 ('June').
- E. `months[6]` returns 'June'.
-

Question 4 (Chapter 2 — Slicing with step)

Consider the following code:

```
sectors = ['Tech', 'Finance', 'Energy', 'Health', 'Consumer', 'Utilities']
subset = sectors[1:5:2]
```

- A. `subset` is equal to `['Finance', 'Health']`.
- B. `sectors[-1]` returns 'Consumer'.
- C. `sectors[:3]` returns `['Tech', 'Finance', 'Energy']`.
- D. `sectors[1:5]` returns four elements.
- E. `sectors[0:6:3]` returns `['Tech', 'Energy', 'Consumer']`.
-

Question 5 (Chapter 2 — Methods and `.sort()`)

Consider the following code:

```
prices = [238.11, 237.81, 238.91]
sorted_prices = prices.sort()
print(sorted_prices)
print(prices)
```

- A. `prices.sort()` modifies `prices` in place.
- B. After `prices.sort()`, `print(prices)` displays `[237.81, 238.11, 238.91]`.
- C. `sorted_prices` contains the sorted list `[237.81, 238.11, 238.91]`.
- D. `print(sorted_prices)` displays `None`.
- E. `.sort()` is a method called on a list object.
-

Question 6 (Chapter 3 — NumPy arrays and element-wise operations)

Consider the following code:

```
import numpy as np
prices = np.array([100, 200, 300])
earnings = np.array([5, 10, 15])
pe_ratios = prices / earnings
print(pe_ratios)
```

- A. `print(pe_ratios)` outputs `[20. 20. 20.]`.
- B. If `prices` and `earnings` were Python lists, `prices / earnings` would perform element-wise division.
- C. `np.mean(pe_ratios)` returns `20.0`.

- D. `pe_ratios` has two dimensions (a 2-D array).
- E. All elements of `pe_ratios` are equal to each other.
-

Question 7 (Chapter 3 — Array shape and `.size`)

Consider the following code:

```
import numpy as np
months = [1, 2, 3]
prices = [238.11, 237.81, 238.91]
cpi_array = np.array([months, prices])
```

- A. `cpi_array.shape` is (2, 3).
- B. `cpi_array.size` is 6.
- C. `cpi_array` has 2 rows and 3 columns.
- D. `cpi_array.shape` is (3, 2).
- E. `cpi_array.ndim` is 3.
-

Question 8 (Chapter 3 / 5 — Boolean filtering and `np.mean()`)

Consider the following code:

```
import numpy as np
stock_prices = np.array([50, 120, 200, 80, 300])
filter_array = stock_prices >= 100
filtered = stock_prices[filter_array]
```

- A. `filter_array` contains False, True, True, False, True in that order.
- B. The variable `filtered` is equal to `np.array([120, 200, 300])`.
- C. `np.mean(filtered)` returns 150.0.
- D. `len(filtered)` returns 5.
- E. Changing the condition to `stock_prices > 200` would make `filtered` equal to `np.array([300])`.
-

Question 9 (Chapter 4 — Matplotlib plotting)

Consider the following code:

```
import matplotlib.pyplot as plt
months = [1, 2, 3, 4, 5, 6]
prices = [238.11, 237.81, 238.91, 239.05, 238.50, 239.20]
plt.plot(months, prices, color='red', linestyle='--')
plt.xlabel('Months')
plt.ylabel('CPI')
plt.title('Monthly CPI')
plt.show()
```

- A. `plt.plot(months, prices, ...)` creates a line plot of prices against months.

- B. The argument `color='red'` sets the line color to red.
 - C. The argument `linestyle='--'` creates a dashed line.
 - D. `plt.xlabel('Months')` sets the title of the plot.
 - E. `plt.title('Monthly CPI')` sets the title of the plot.
-

Question 10 (Chapter 5 – S&P 100 case study: P/E ratios and histograms)

Consider the following code:

```
import numpy as np
import matplotlib.pyplot as plt
prices = np.array([100, 150, 200, 120])
earnings = np.array([5, 10, 20, 4])
pe_ratios = prices / earnings
average = np.mean(pe_ratios)
plt.hist(x=pe_ratios, bins=4)
plt.show()
```

- A. `pe_ratios` is computed by element-wise division of prices by earnings.
 - B. `pe_ratios[0]` is equal to 20.0.
 - C. `average` is equal to 15.0.
 - D. `plt.hist(x=pe_ratios, bins=4)` creates a histogram of the P/E ratios.
 - E. Increasing bins from 4 to 8 would divide the data into more bins.
-

Question 11 (Review – NumPy type coercion vs lists)

Consider the following code:

```
import numpy as np
mixed = np.array([3, 'is', True])
print(mixed[0])
print(type(mixed[0]))
```

- A. All elements of `mixed` are stored as strings.
 - B. `mixed[0]` is the integer 3.
 - C. `mixed[0] == 3` (comparison to the integer 3) is True.
 - D. `np.array([3, 'is', True])` raises a `TypeError` because the types are incompatible.
 - E. A Python list `[3, 'is', True]` can hold mixed types without conversion.
-

Question 12 (Review – `.sort()` in place and list references)

Consider the following code:

```
prices = [238.91, 237.81, 238.11]
original = prices
result = prices.sort()
```

- A. `result` is `None`.
- B. After `prices.sort()`, `prices` is `[237.81, 238.11, 238.91]`.
- C. `original` is still `[238.91, 237.81, 238.11]`.
- D. `original` and `prices` refer to the same list object.
- E. `prices.sort()` modifies the list in place.